

BLOCKCHAIN ENGINEERING

ASSIGNMENT - 012

Name: Aman Sarkar

Ht.no: 2303A51273

Batch: 05

Code -

```
Assignment-12.py > ...
1  import hashlib
2  import time
3  from tkinter import *
4
5  # -----
6  # SMART CONTRACT SIMULATION (ASSET PORTFOLIO)
7
8  class PortfolioContract:
9      def __init__(self):
10         self.assets = {}          # store assets
11         self.transactions = []    # transaction history
12
13     def create_asset(self, asset_id, owner, value):
14         timestamp = time.strftime("%Y-%m-%d %H:%M:%S")
15
16         asset = {
17             "id": asset_id,
18             "owner": owner,
19             "value": value
20         }
21
22         self.assets[asset_id] = asset
23
24         tx_hash = self.generate_hash(asset, timestamp)
25
26         tx = {
27             "action": "CREATE",
28             "asset": asset,
29             "time": timestamp,
30             "hash": tx_hash
31         }
32
33         self.transactions.append(tx)
34         return tx
35
36     def get_all_assets(self):
37         return list(self.assets.values())
38
39     def transfer_asset(self, asset_id, new_owner):
```

```

39     def transfer_asset(self, asset_id, new_owner):
40         if asset_id not in self.assets:
41             return None
42
43         timestamp = time.strftime("%Y-%m-%d %H:%M:%S")
44
45         self.assets[asset_id]["owner"] = new_owner
46
47         tx_hash = self.generate_hash(self.assets[asset_id], timestamp)
48
49         tx = {
50             "action": "TRANSFER",
51             "asset": self.assets[asset_id],
52             "time": timestamp,
53             "hash": tx_hash
54         }
55
56         self.transactions.append(tx)
57         return tx
58
59     def generate_hash(self, data, timestamp):
60         return hashlib.sha256(f"{data}{timestamp}".encode()).hexdigest()
61
62
63     # -----
64     # CONTRACT OBJECT
65     contract = PortfolioContract()
66
67     # -----
68     # GUI FUNCTIONS
69
70     def create_asset():
71         asset_id = entry_id.get()
72         owner = entry_owner.get()
73         value = entry_value.get()
74
75         if asset_id and owner and value:
76             tx = contract.create_asset(asset_id, owner, value)

```

```

78         status_label.config(text="Asset Created ✓", fg="green")
79         hash_label.config(text=f"Hash:\n{tx['hash']}")
80
81         update_assets()
82         update_history()
83     else:
84         status_label.config(text="Fill all fields!", fg="red")
85
86
87 def show_assets():
88     assets = contract.get_all_assets()
89
90     text = "Assets:\n"
91     for a in assets:
92         text += f"{a['id']} → {a['owner']} ({a['value']})\n"
93
94     assets_label.config(text=text)
95
96
97 def transfer_asset():
98     asset_id = entry_id.get()
99     new_owner = entry_owner.get()
100
101     tx = contract.transfer_asset(asset_id, new_owner)
102
103     if tx:
104         status_label.config(text="Ownership Transferred ✓", fg="blue")
105         hash_label.config(text=f"Hash:\n{tx['hash']}")
106
107         update_assets()
108         update_history()
109     else:
110         status_label.config(text="Asset not found!", fg="red")
111
112
113 def update_assets():
114     show_assets()

```

```

117 def update_history():
118     text = "Transaction History:\n"
119     for tx in contract.transactions:
120         text += f"{tx['time']} [{tx['action']}] {tx['asset']['id']} → {tx['asset']['owner']}\n"
121
122     history_label.config(text=text)
123
124
125 # -----
126 # GUI SETUP
127
128 root = Tk()
129 root.title("Asset Portfolio (Blockchain Simulation)")
130
131 Label(root, text="Asset ID").pack()
132 entry_id = Entry(root)
133 entry_id.pack()
134
135 Label(root, text="Owner").pack()
136 entry_owner = Entry(root)
137 entry_owner.pack()
138
139 Label(root, text="Value").pack()
140 entry_value = Entry(root)
141 entry_value.pack()
142
143 Button(root, text="Create Asset", command=create_asset).pack(pady=5)
144 Button(root, text="View Assets", command=show_assets).pack(pady=5)
145 Button(root, text="Transfer Ownership", command=transfer_asset).pack(pady=5)
146
147 status_label = Label(root, text="")
148 status_label.pack()
149
150 hash_label = Label(root, text="")
151 hash_label.pack()
152
153 assets_label = Label(root, text="Assets:")
154 assets_label.pack()

```

```

156 history_label = Label(root, text="Transaction History:")
157 history_label.pack()
158
159 root.mainloop()

```

Output –

